

2-DOF CubeMars Arm – Trajectory Control

Controller survey: from joint-space PD to Computed Torque
Implementation via `ak_v3_bus.py` MIT mode

Repo `exo-actuator-sensor-drivers`

May 21, 2026

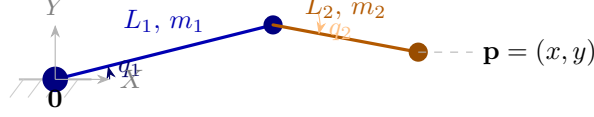
Contents

1	System Description	2
1.1	Geometric parameters	2
2	Robot Model	2
2.1	Forward kinematics	2
2.2	Jacobian	2
2.3	Equations of motion	2
3	Available CubeMars Control Modes	3
4	Trajectory Generation	3
4.1	Inverse kinematics (closed-form, 2-DOF)	3
4.2	Joint velocity and acceleration from Jacobian	3
4.3	Minimum-jerk trajectory in Cartesian space	3
5	Controllers	4
5.1	Joint-space PD	4
5.2	Joint-space PD with Gravity Compensation	4
5.3	Joint-space PID	4
5.4	Computed Torque Control (CTC)	5
5.5	CTC with Inner MIT Damping	6
5.6	Cartesian PD (Jacobian Transpose)	6
5.7	Operational Space Control (Task-Space Inversion)	7
5.8	Cartesian Impedance Control	8
6	Two-Motor Implementation Architecture	8
6.1	Bus setup: two AkV3Bench instances	8
6.2	Control loop skeleton	9
6.3	Feedback latency and timing	9
7	Controller Comparison	10
8	Worked Example: Circle Trajectory with CTC	10
9	Safety Constraints	12

1 System Description

The bench robot is a **2-DOF planar arm** driven by two **CubeMars AK60-6 V3.0 KV80** actuators. The shoulder motor (q_1) and elbow motor (q_2) are each connected to a SocketCAN bus (`can1`) via the DSDTech SH-C30A adapter and controlled through the `ak.v3_bus.py` / `AK_V3_CAN.py` driver stack.

1.1 Geometric parameters



Symbol	Meaning	Typical value
L_1	Upper-arm link length	0.30 m
L_2	Forearm link length	0.28 m
m_1	Upper-arm mass (+ motor)	1.8 kg
m_2	Forearm mass (+ motor)	1.2 kg
I_1, I_2	Link moments of inertia	$m_i L_i^2 / 3$ (uniform rod)
g	Gravitational acceleration	9.81 m/s^2
K_t	AK60-6 motor torque constant	$\approx 0.14 \text{ N}\cdot\text{m/A}$

2 Robot Model

2.1 Forward kinematics

Let $s_1 = \sin q_1$, $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$, $c_{12} = \cos(q_1 + q_2)$. The end-effector Cartesian position is

$$\mathbf{p}(\mathbf{q}) = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} L_1 c_1 + L_2 c_{12} \\ L_1 s_1 + L_2 s_{12} \end{bmatrix}. \quad (1)$$

2.2 Jacobian

The geometric Jacobian relates joint velocities to end-effector velocity: $\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$,

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}. \quad (2)$$

The Jacobian is singular when links are fully extended ($q_2 = 0$ or π). Damped pseudoinverse $\mathbf{J}^\dagger = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1}$ is used near singularities ($\lambda \approx 0.01$ – 0.05).

2.3 Equations of motion

The standard rigid-body equations of motion are

$$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}, \quad (3)$$

where $\boldsymbol{\tau} = [\tau_1, \tau_2]^T$ are joint torques delivered by the motors (output-shaft, N·m).

Inertia matrix. Treating each link as a uniform rod of mass m_i and length L_i (so $I_i = m_i L_i^2 / 3$), with $c_2 = \cos q_2$:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} M_{11} & M_{12} \\ M_{12} & M_{22} \end{bmatrix}, \quad \begin{aligned} M_{11} &= \frac{4}{3} m_1 L_1^2 + m_2 (L_1^2 + \frac{4}{3} L_2^2 + 2 L_1 L_2 c_2), \\ M_{12} &= m_2 (\frac{4}{3} L_2^2 + L_1 L_2 c_2), \\ M_{22} &= \frac{4}{3} m_2 L_2^2. \end{aligned} \quad (4)$$

Coriolis/centrifugal matrix. With $s_2 = \sin q_2$ and $h = -m_2 L_1 L_2 s_2$:

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} h\dot{q}_2 & h(\dot{q}_1 + \dot{q}_2) \\ -h\dot{q}_1 & 0 \end{bmatrix}. \quad (5)$$

Gravity vector. (Vertical plane, Y upward.)

$$\mathbf{g}(\mathbf{q}) = g \begin{bmatrix} (m_1/2 + m_2) L_1 c_1 + m_2 L_2/2 c_{12} \\ m_2 L_2/2 c_{12} \end{bmatrix}. \quad (6)$$

3 Available CubeMars Control Modes

The AK60-6 V3.0 firmware exposes the following modes, all reachable through `AK_V3_CAN.AkV3Motor` and the `ak_v3.bus.AkV3Bench` wrapper.

Mode	Packet type	data_name	Signal sent
MIT (force-control)	0x08	goal_position / goal_torque	$K_p, K_d, q_d, \dot{q}_d, \tau_{\text{ff}}$
Position loop	0x04	goal_position_deg	q_d (degrees)
Velocity loop	0x03	goal_speed_erpm	ω_d (ERPM)
Current loop	0x01	goal_duty	I_q (duty)
Pos-vel profile	0x06	goal_pos_spd	q_d, ω_d, α

The **MIT mode** is the most powerful because it lets the *host* choose the torque it commands:

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + \tau_{\text{ff}}, \quad (7)$$

with $K_p \in [0, 500]$ and $K_d \in [0, 5]$ sent in the same CAN frame as the setpoint. Setting $K_p = K_d = 0$ delivers a *pure torque* τ_{ff} directly from the current loop – the fundamental building block for all model-based controllers described below.

4 Trajectory Generation

A *prescribed* end-effector trajectory $\mathbf{p}_d(t), \dot{\mathbf{p}}_d(t), \ddot{\mathbf{p}}_d(t)$ must be converted to joint-space references $\mathbf{q}_d(t), \dot{\mathbf{q}}_d(t), \ddot{\mathbf{q}}_d(t)$ before most joint-space controllers can be applied.

4.1 Inverse kinematics (closed-form, 2-DOF)

$$q_2 = \text{atan2}\left(\pm\sqrt{1 - D^2}, D\right), \quad D = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}, \quad (8)$$

$$q_1 = \text{atan2}(y, x) - \text{atan2}(L_2 s_2, L_1 + L_2 c_2). \quad (9)$$

The $\pm$ sign selects elbow-up (+) or elbow-down (−) configuration.

4.2 Joint velocity and acceleration from Jacobian

Differentiate (1):

$$\dot{\mathbf{q}}_d = \mathbf{J}^\dagger(\mathbf{q}_d) \dot{\mathbf{p}}_d, \quad \ddot{\mathbf{q}}_d = \mathbf{J}^\dagger(\mathbf{q}_d) (\ddot{\mathbf{p}}_d - \dot{\mathbf{J}}(\mathbf{q}_d, \dot{\mathbf{q}}_d) \dot{\mathbf{q}}_d). \quad (10)$$

4.3 Minimum-jerk trajectory in Cartesian space

For point-to-point moves from $\mathbf{p}_0$ to $\mathbf{p}_f$ in time T , the minimum-jerk polynomial satisfies zero position, velocity, and acceleration at both endpoints:

$$\mathbf{p}_d(t) = \mathbf{p}_0 + (\mathbf{p}_f - \mathbf{p}_0) \left[10(t/T)^3 - 15(t/T)^4 + 6(t/T)^5 \right]. \quad (11)$$

5 Controllers

All controllers are compared in Table ?? at the end of this section.

5.1 Joint-space PD

Idea. Ignore robot dynamics; treat each joint independently as a spring-damper.

$$\boldsymbol{\tau} = \mathbf{K}_p \mathbf{e} + \mathbf{K}_d \dot{\mathbf{e}}, \quad \mathbf{e} = \mathbf{q}_d - \mathbf{q}, \quad \dot{\mathbf{e}} = \dot{\mathbf{q}}_d - \dot{\mathbf{q}}. \quad (12)$$

$\mathbf{K}_p = \text{diag}(k_{p1}, k_{p2})$ and $\mathbf{K}_d = \text{diag}(k_{d1}, k_{d2})$ are diagonal positive-definite gain matrices. For stable regulation choose $k_{pi} > 0$, $k_{di} > 0$.

MIT mode mapping. The motor's own on-board PD loop (7) *already is* this controller when $\tau_{ff} = 0$. Set the MIT gains directly:

Listing 1: Joint-space PD via on-board MIT gains

```
1 # shoulder = j1, elbow = j2
2 bus1.write("goal_position", {"j1": q_d1}, kp=60, kd=1.5)
3 bus2.write("goal_position", {"j2": q_d2}, kp=40, kd=1.2)
```

This works but ignores cross-coupling between joints. Suitable for low-speed, lightly loaded motions.

5.2 Joint-space PD with Gravity Compensation

Add the static gravity term to remove steady-state error:

$$\boldsymbol{\tau} = \mathbf{K}_p \mathbf{e} + \mathbf{K}_d \dot{\mathbf{e}} + \mathbf{g}(\mathbf{q}). \quad (13)$$

In MIT mode, pass the gravity torques as feedforward:

Listing 2: PD + gravity compensation

```
1 import numpy as np
2
3 def gravity(q, L, m, g=9.81):
4     g1 = g*((m[0]/2 + m[1])*L[0]*np.cos(q[0])
5             + m[1]*L[1]/2*np.cos(q[0]+q[1]))
6     g2 = g*m[1]*L[1]/2*np.cos(q[0]+q[1])
7     return np.array([g1, g2])
8
9 # Inside control loop:
10 q, dq, _ = bus1.read_state()["j1"], ... # fetch state
11 tau_g = gravity(q, L, m)
12
13 bus1.write("goal_position", {"j1": q_d[0]},
14           kp=60, kd=1.5, tau_ff=tau_g[0])
15 bus2.write("goal_position", {"j2": q_d[1]},
16           kp=40, kd=1.2, tau_ff=tau_g[1])
```

5.3 Joint-space PID

Add an integral term to eliminate residual steady-state error:

$$\boldsymbol{\tau} = \mathbf{K}_p \mathbf{e} + \mathbf{K}_i \int_0^t \mathbf{e} d\tau + \mathbf{K}_d \dot{\mathbf{e}}. \quad (14)$$

Anti-windup: clamp the integral $\|\mathbf{e}_I\|_\infty \leq e_I^{\max}$ each tick. Send the computed torque entirely as τ_{ff} (set $K_p = K_d = 0$ in the MIT frame) so on-board gains do not double-count:

Listing 3: Software PID (tau_ff only)

```

1 e_int = np.zeros(2) # persistent integral accumulator
2
3 def pid_step(q, dq, q_d, dqd, dt, Kp, Ki, Kd, e_int,
4             e_int_max=0.5):
5     e = q_d - q
6     de = dqd - dq
7     e_int += e * dt
8     e_int = np.clip(e_int, -e_int_max, e_int_max) # anti-windup
9     return Kp*e + Ki*e_int + Kd*de, e_int
10
11 # Usage (kp=kd=0: pure tau_ff)
12 tau, e_int = pid_step(q, dq, q_d, dqd, dt,
13                      Kp=np.array([40,30]),
14                      Ki=np.array([5, 3]),
15                      Kd=np.array([1.5,1.0]), e_int=e_int)
16 bus1.write("goal_torque", {"j1": tau[0]})
17 bus2.write("goal_torque", {"j2": tau[1]})

```

5.4 Computed Torque Control (CTC)

Idea. Invert the full nonlinear dynamics so the closed-loop error behaves as a *linear* second-order system:

$$\tau = \mathbf{M}(\mathbf{q})[\ddot{\mathbf{q}}_d + \mathbf{K}_d \dot{\mathbf{e}} + \mathbf{K}_p \mathbf{e}] + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}). \quad (15)$$

Substituting into (3) gives the linear error dynamics:

$$\ddot{\mathbf{q}}_e + \mathbf{K}_d \dot{\mathbf{q}}_e + \mathbf{K}_p \mathbf{q}_e = \mathbf{0}, \quad (16)$$

where $\mathbf{q}_e = \mathbf{q}_d - \mathbf{q}$. Choose $\mathbf{K}_d = 2\zeta\omega_n \mathbf{I}$ and $\mathbf{K}_p = \omega_n^2 \mathbf{I}$ for critically damped response at bandwidth ω_n [rad/s].

Practical bandwidth. The full CTC loop must close at $\geq 5\omega_n$. With the CubeMars CAN round-trip ≈ 1 ms (1 kHz effective), target $\omega_n \leq 20$ rad/s.

Listing 4: Computed Torque Control implementation

```

1 import numpy as np
2
3 # --- Robot parameters (fill in your measured values) ---
4 L = np.array([0.30, 0.28]) # link lengths [m]
5 m = np.array([1.80, 1.20]) # link masses [kg]
6 g = 9.81
7
8 def inertia_matrix(q):
9     c2 = np.cos(q[1])
10    M11 = (4/3*m[0]*L[0]**2
11          + m[1]*(L[0]**2 + 4/3*L[1]**2 + 2*L[0]*L[1]*c2))
12    M12 = m[1]*(4/3*L[1]**2 + L[0]*L[1]*c2)
13    M22 = 4/3*m[1]*L[1]**2
14    return np.array([[M11, M12], [M12, M22]])
15
16 def coriolis_matrix(q, dq):
17     h = -m[1]*L[0]*L[1]*np.sin(q[1])
18     return np.array([[h*dq[1], h*(dq[0]+dq[1])],
19                     [-h*dq[0], 0.0 ]])
20

```

```

21 def gravity_vector(q):
22     g1 = g*((m[0]/2+m[1])*L[0]*np.cos(q[0])
23             + m[1]*L[1]/2*np.cos(q[0]+q[1]))
24     g2 = g*m[1]*L[1]/2*np.cos(q[0]+q[1])
25     return np.array([g1, g2])
26
27 # --- CTC gains (critically damped, omega_n = 15 rad/s) ---
28 wn = 15.0
29 Kp = wn**2 * np.eye(2)
30 Kd = 2*wn * np.eye(2) # zeta = 1
31
32 # --- Control loop (100 Hz) ---
33 def ctc_torque(q, dq, q_d, dqd, qdd_d):
34     e = q_d - q
35     de = dqd - dq
36     v = qdd_d + Kd @ de + Kp @ e # desired acceleration
37     tau = inertia_matrix(q) @ v \
38           + coriolis_matrix(q, dq) @ dq \
39           + gravity_vector(q)
40     return np.clip(tau, -12.0, 12.0) # AK60-6 T_MAX = 12 N.m
41
42 # Send as pure feedforward (kp=kd=0 bypasses on-board PD):
43 tau = ctc_torque(q, dq, q_d, dqd, qdd_d)
44 bus1.write("goal_torque", {"j1": tau[0]})
45 bus2.write("goal_torque", {"j2": tau[1]})

```

Robustness note. CTC performance degrades when the model parameters (m_i, L_i) are inaccurate. Add a PD feedforward term in the MIT frame as an outer stabilizing loop (Section 5.5).

5.5 CTC with Inner MIT Damping

A practical hybrid: CTC provides the model-inversion feedforward while small on-board K_d damps unmodelled high-frequency oscillations without fighting the CTC loop:

$$\tau_{\text{ff}} = \mathbf{M}(\mathbf{q})[\ddot{\mathbf{q}}_d + \mathbf{K}_d^{\text{outer}} \dot{\mathbf{e}} + \mathbf{K}_p^{\text{outer}} \mathbf{e}] + \mathbf{C}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}), \quad (17)$$

$$\text{CAN frame: } K_p^{\text{MIT}} = 0, \quad K_d^{\text{MIT}} = 0.3\text{--}0.6, \quad \tau_{\text{ff}} = \text{eq. (17)}. \quad (18)$$

Listing 5: Hybrid CTC – feedforward + small on-board damping

```

1 tau = ctc_torque(q, dq, q_d, dqd, qdd_d)
2 # kp=0 (no spring), kd=0.4 (damps encoder noise), tau_ff = CTC output
3 bus1.write("mit_command", {"j1": q_d[0]},
4             kp=0.0, kd=0.4, dq_des=dqd[0], tau_ff=tau[0])
5 bus2.write("mit_command", {"j2": q_d[1]},
6             kp=0.0, kd=0.4, dq_des=dqd[1], tau_ff=tau[1])

```

5.6 Cartesian PD (Jacobian Transpose)

Apply spring-damper forces in *Cartesian space* and back-project through the Jacobian transpose:

$$\boldsymbol{\tau} = \mathbf{J}^T(\mathbf{q})[\mathbf{K}_p^{\text{cart}}(\mathbf{p}_d - \mathbf{p}) + \mathbf{K}_d^{\text{cart}}(\dot{\mathbf{p}}_d - \dot{\mathbf{p}})] + \mathbf{g}(\mathbf{q}). \quad (19)$$

No matrix inversion needed; avoids singularity issues present in pseudoinverse methods (near singularity the transpose simply produces less Cartesian force, which is safe).

Listing 6: Cartesian PD via Jacobian transpose

```

1 def jacobian(q):
2     s1 = np.sin(q[0]); c1 = np.cos(q[0])
3     s12 = np.sin(q[0]+q[1]); c12 = np.cos(q[0]+q[1])
4     return np.array([
5         [-L[0]*s1 - L[1]*s12, -L[1]*s12],
6         [ L[0]*c1 + L[1]*c12, L[1]*c12]
7     ])
8
9 def fk(q):
10    return np.array([
11        L[0]*np.cos(q[0]) + L[1]*np.cos(q[0]+q[1]),
12        L[0]*np.sin(q[0]) + L[1]*np.sin(q[0]+q[1])
13    ])
14
15 Kpc = np.diag([200, 200]) # N/m Cartesian stiffness
16 Kdc = np.diag([ 20, 20]) # N.s/m Cartesian damping
17
18 def cartpd_torque(q, dq, p_d, pd_d):
19     p = fk(q)
20     J = jacobian(q)
21     pd = J @ dq # current EE velocity
22     F = Kpc @ (p_d - p) + Kdc @ (pd_d - pd)
23     return J.T @ F + gravity_vector(q)
24
25 tau = cartpd_torque(q, dq, p_d, pd_d)
26 bus1.write("goal_torque", {"j1": tau[0]})
27 bus2.write("goal_torque", {"j2": tau[1]})

```

5.7 Operational Space Control (Task-Space Inversion)

Full model-based control directly in task space. Define the *operational-space inertia matrix* $\Lambda = (\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^T)^{-1}$, Coriolis-like term $\boldsymbol{\mu} = \Lambda\mathbf{J}\mathbf{M}^{-1}\mathbf{C}\dot{\mathbf{q}}$, and gravity projection $\boldsymbol{\rho} = \Lambda\mathbf{J}\mathbf{M}^{-1}\mathbf{g}$:

$$\boldsymbol{\tau} = \mathbf{J}^T [\Lambda(\ddot{\mathbf{p}}_d + \mathbf{K}_d^{\text{OS}} \dot{\mathbf{e}}^{\text{cart}} + \mathbf{K}_p^{\text{OS}} \mathbf{e}^{\text{cart}}) + \boldsymbol{\mu} + \boldsymbol{\rho}], \quad (20)$$

where $\mathbf{e}^{\text{cart}} = \mathbf{p}_d - \mathbf{p}$ and $\dot{\mathbf{e}}^{\text{cart}} = \dot{\mathbf{p}}_d - \dot{\mathbf{p}}$. The closed-loop task-space dynamics are linear: $\ddot{\mathbf{p}}_e + \mathbf{K}_d^{\text{OS}} \dot{\mathbf{p}}_e + \mathbf{K}_p^{\text{OS}} \mathbf{p}_e = \mathbf{0}$.

Listing 7: Operational Space Control

```

1 def osc_torque(q, dq, p_d, pd_d, pdd_d):
2     J = jacobian(q); M = inertia_matrix(q)
3     C = coriolis_matrix(q, dq); gv = gravity_vector(q)
4     Minv = np.linalg.inv(M)
5
6     lam_inv = J @ Minv @ J.T
7     lam = np.linalg.inv(lam_inv + 1e-4*np.eye(2)) # damped
8
9     mu = lam @ J @ Minv @ C @ dq
10    rho = lam @ J @ Minv @ gv
11
12    p = fk(q)
13    pd = J @ dq
14    e_c = p_d - p
15    de_c = pd_d - pd
16
17    Kp_os = 400*np.eye(2) # N/m
18    Kd_os = 40*np.eye(2) # N.s/m (critically damped at ~10 Hz)

```

```

19     F_os = lam @ (pdd_d + Kd_os @ de_c + Kp_os @ e_c) + mu + rho
20     return np.clip(J.T @ F_os, -12.0, 12.0)
21
22
23 tau = osc_torque(q, dq, p_d, pd_d, pdd_d)
24 bus1.write("goal_torque", {"j1": tau[0]})
25 bus2.write("goal_torque", {"j2": tau[1]})

```

5.8 Cartesian Impedance Control

Impedance control shapes the *apparent mechanical impedance* of the end-effector – useful for compliant interaction with the environment:

$$\mathcal{M}_d \ddot{\mathbf{e}}_{dd}^{\text{cart}} + \mathbf{B}_d \dot{\mathbf{e}}^{\text{cart}} + \mathbf{K}_d \mathbf{e}^{\text{cart}} = \mathbf{f}_{\text{ext}}, \quad (21)$$

where $\mathcal{M}_d$, $\mathbf{B}_d$, $\mathbf{K}_d$ are the desired Cartesian inertia, damping, and stiffness tensors and $\mathbf{f}_{\text{ext}}$ is the external contact force. Setting $\mathbf{f}_{\text{ext}} = \mathbf{0}$ (no contact) reduces to the trajectory-tracking case.

Implementation via task-space torque:

$$\boldsymbol{\tau} = \mathbf{J}^T [\mathbf{K}_d(\mathbf{p}_d - \mathbf{p}) + \mathbf{B}_d(\dot{\mathbf{p}}_d - \dot{\mathbf{p}}) + \mathcal{M}_d(\ddot{\mathbf{p}}_d - \mathbf{J}\mathbf{M}^{-1}(\mathbf{C}\dot{\mathbf{q}} + \mathbf{g} - \boldsymbol{\tau}_{\text{null}}))] + \mathbf{g}(\mathbf{q}), \quad (22)$$

For free-space tracking the simplified form is equivalent to Cartesian PD (19) with $\mathbf{K}_p^{\text{cart}} \rightarrow \mathbf{K}_d$ and $\mathbf{K}_d^{\text{cart}} \rightarrow \mathbf{B}_d$.

Listing 8: Cartesian impedance (free-space tracking)

```

1  # Desired impedance parameters
2  Md = np.diag([1.0, 1.0]) # kg -- virtual mass
3  Bd = np.diag([30, 30]) # N.s/m -- virtual damping
4  Kd = np.diag([300, 300]) # N/m -- virtual stiffness
5
6  def impedance_torque(q, dq, p_d, pd_d, pdd_d):
7      J = jacobian(q)
8      p = fk(q); pd = J @ dq
9      e = p_d - p; de = pd_d - pd
10
11     F_imp = Kd @ e + Bd @ de + Md @ pdd_d # desired task force
12     return np.clip(J.T @ F_imp + gravity_vector(q), -12.0, 12.0)
13
14 tau = impedance_torque(q, dq, p_d, pd_d, pdd_d)
15 bus1.write("goal_torque", {"j1": tau[0]})
16 bus2.write("goal_torque", {"j2": tau[1]})

```

The MIT mode can also implement joint-space impedance natively by setting K_p^{MIT} and K_d^{MIT} to low values and updating q_d each tick, as in the kinesthetic teaching scripts `08_kinesthetic_record.py` / `05_kinesthetic_record.py`.

6 Two-Motor Implementation Architecture

6.1 Bus setup: two AkV3Bench instances

Each motor lives on its own AkV3Bench (separate `can_id`). Both share the same SocketCAN bus handle:

Listing 9: Dual-motor setup

```

1  import can
2  from ak_v3_bus import AkV3Bench
3

```

```

4 # Open one SocketCAN bus shared by both motors
5 raw_bus = can.Bus(channel="can1", interface="socketcan")
6
7 shoulder = AkV3Bench(channel="can1", can_id=0x01,
8                     model="AK60-6", name="shoulder")
9 elbow = AkV3Bench(channel="can1", can_id=0x02,
10                  model="AK60-6", name="elbow")
11
12 shoulder.connect(set_zero=True)
13 elbow.connect( set_zero=True)

```

6.2 Control loop skeleton

Listing 10: Generic 100-Hz control loop

```

1 import time
2 import numpy as np
3
4 DT = 0.01 # 100 Hz control loop
5 t0 = time.monotonic()
6
7 try:
8     while True:
9         t = time.monotonic() - t0
10
11         # 1. Read joint state (one CAN round-trip each)
12         q1, dq1, _ = shoulder.read_state()["shoulder"]
13         q2, dq2, _ = elbow.read_state()["elbow"]
14         q = np.array([q1, q2 ])
15         dq = np.array([dq1, dq2])
16
17         # 2. Evaluate reference trajectory at time t
18         q_d, dqd, qdd_d = trajectory(t) # user-defined function
19
20         # 3. Compute control torques (select one controller below)
21         tau = ctc_torque(q, dq, q_d, dqd, qdd_d) # CTC
22         # tau = cartpd_torque(q, dq, p_d(t), pd_d(t)) # Cartesian PD
23         # tau = osc_torque(q, dq, p_d(t), pd_d(t), pdd_d(t)) # OSC
24
25         # 4. Send commands (pure feedforward)
26         shoulder.write("goal_torque", {"shoulder": tau[0]})
27         elbow.write( "goal_torque", {"elbow": tau[1]})
28
29         # 5. Pace loop
30         time.sleep(max(0, DT - (time.monotonic() - t0 - t)))
31
32 except KeyboardInterrupt:
33     pass
34 finally:
35     shoulder.disconnect()
36     elbow.disconnect()

```

6.3 Feedback latency and timing

Each `read_state()` call triggers one MIT frame (to provoke a reply in Query-Reply CAN mode) and blocks for up to 100 ms. For a two-joint arm this adds ≈ 2 ms serialised latency per control tick. Mitigations:

- Use **Periodic Feedback** mode (set via R-Link): motor auto-broadcasts at a fixed rate. Parse frames in a background thread and read the latest cached state.
- Send both MIT commands *before* reading feedback to reduce effective round-trip time.
- For CTC at $\omega_n \leq 15$ rad/s a 10 ms loop with 2 ms feedback latency introduces $< 10\%$ phase margin loss – still stable.

7 Controller Comparison

Controller	Model needed	Singularity handling	Steady-state error	Best use case
Joint-space PD	None	N/A (joint)	Yes (with gravity)	Fast prototyping
PD + gravity comp.	$\mathbf{g}(\mathbf{q})$ only	N/A	Eliminated	Slow point-to-point
PID	None	N/A	Eliminated	Unknown disturbances
CTC	Full $\mathbf{M}, \mathbf{C}, \mathbf{g}$	N/A	Eliminated	Trajectory tracking
CTC + inner damping	Full model	N/A	Eliminated	Noisy encoders / flexible links
Cartesian PD (J transpose)	$\mathbf{g} + \mathbf{J}$	Safe (no inversion)	Yes (joint space)	Soft interaction
Operational Space Control	Full model	Damped pseudo-inv	Eliminated	Cartesian trajectory
Cartesian impedance	$\mathbf{J} + \mathbf{g}$	Safe	Designed-in	Contact / teaching

8 Worked Example: Circle Trajectory with CTC

Generate a circular end-effector trajectory of radius $r = 0.05$ m centred at $(0.40, 0.15)$ m, period $T = 4$ s, then control it with CTC.

Listing 11: Circular trajectory + CTC end-to-end

```

1 import numpy as np, time
2 from ak_v3_bus import AkV3Bench
3
4 # ---- Arm parameters ----
5 L = np.array([0.30, 0.28])
6 m = np.array([1.80, 1.20])
7 p_c = np.array([0.40, 0.15]) # circle centre
8 r, T_circ = 0.05, 4.0
9
10 # ---- Trajectory ----
11 def circle(t):
12     th = 2*np.pi*t / T_circ
13     p = p_c + r*np.array([ np.cos(th), np.sin(th)])
14     pd = r*2*np.pi/T_circ*np.array([-np.sin(th), np.cos(th)])
15     pdd = -r*(2*np.pi/T_circ)**2*np.array([np.cos(th), np.sin(th)])
16     return p, pd, pdd
17
18 # ---- IK ----
19 def ik(p):
20     D = (p[0] - L[0]**2 - L[1]**2) / (2*L[0]*L[1])
21     D = np.clip(D, -1+1e-6, 1-1e-6)
22     q2 = np.arctan2(np.sqrt(1-D**2), D) # elbow-up
23     q1 = np.arctan2(p[1], p[0]) - np.arctan2(L[1]*np.sin(q2),
24                                                L[0]+L[1]*np.cos(q2))
25     return np.array([q1, q2])

```

```

26
27 def jacobian(q):
28     s12 = np.sin(q[0]+q[1]); c12 = np.cos(q[0]+q[1])
29     s1 = np.sin(q[0]); c1 = np.cos(q[0])
30     return np.array([[-L[0]*s1-L[1]*s12, -L[1]*s12],
31                     [ L[0]*c1+L[1]*c12, L[1]*c12]])
32
33 def traj_joint(t):
34     p, pd, pdd = circle(t)
35     q_d = ik(p)
36     J = jacobian(q_d)
37     Jinv = np.linalg.pinv(J)
38     qd = Jinv @ pd
39     qdd = Jinv @ pdd # approximate (ignores dJ term at low speed)
40     return q_d, qd, qdd
41
42 # ---- CTC (from Section 4.4) -- inlined ----
43 wn = 12.0
44 Kp_c = wn**2 * np.eye(2)
45 Kd_c = 2*wn * np.eye(2)
46
47 def M(q):
48     c2 = np.cos(q[1])
49     m11 = 4/3*m[0]*L[0]**2 + m[1]*(L[0]**2+4/3*L[1]**2+2*L[0]*L[1]*c2)
50     m12 = m[1]*(4/3*L[1]**2+L[0]*L[1]*c2)
51     return np.array([[m11,m12],[m12,4/3*m[1]*L[1]**2]])
52
53 def C(q,dq):
54     h = -m[1]*L[0]*L[1]*np.sin(q[1])
55     return np.array([[h*dq[1], h*(dq[0]+dq[1])],[-h*dq[0],0.0]])
56
57 def gv(q):
58     g1 = 9.81*((m[0]/2+m[1])*L[0]*np.cos(q[0])+m[1]*L[1]/2*np.cos(q[0]+q[1]))
59     return np.array([g1, 9.81*m[1]*L[1]/2*np.cos(q[0]+q[1])])
60
61 def ctc(q, dq, q_d, dqd, qdd_d):
62     e = q_d-q; de = dqd-dq
63     v = qdd_d + Kd_c@de + Kp_c@e
64     return np.clip(M(q)@v + C(q,dq)@dq + gv(q), -12.0, 12.0)
65
66 # ---- Main loop ----
67 shoulder = AkV3Bench(can_id=0x01, name="shoulder")
68 elbow = AkV3Bench(can_id=0x02, name="elbow")
69
70 with shoulder, elbow:
71     t0 = time.monotonic()
72     while True:
73         t = time.monotonic()-t0
74         q1,dq1,_ = shoulder.read_state()["shoulder"]
75         q2,dq2,_ = elbow.read_state()["elbow"]
76         q = np.array([q1,q2]); dq = np.array([dq1,dq2])
77         q_d, dqd, qdd_d = traj_joint(t)
78         tau = ctc(q, dq, q_d, dqd, qdd_d)
79         shoulder.write("goal_torque", {"shoulder": tau[0]})
80         elbow.write("goal_torque", {"elbow": tau[1]})
81         time.sleep(0.01)

```

9 Safety Constraints

- **Torque limits:** $|\tau_i| \leq 12 \text{ N}\cdot\text{m}$ (`T_MAX`). Always clip computed torques before sending.
- **Velocity limits:** $|\dot{q}_i| \leq 60 \text{ rad/s}$. Saturate $\dot{q}_d$ in the trajectory generator.
- **Singularity avoidance:** $|q_2| > 5^\circ$ from fully extended / folded. Monitor $|\det \mathbf{J}|$; switch to Cartesian PD (J-transpose) near singularities.
- **Workspace limits:** Impose joint-angle limits in IK: $q_1 \in [-\pi/2, \pi]$, $q_2 \in [10^\circ, 170^\circ]$ (hardware stops).
- **Emergency stop:** Send `bus.write("goal_torque", {name: 0.0})` on Ctrl+C (handled by `__exit__` in `AkV3Bench`).
- **Thermal:** Monitor `read_raw()["shoulder"][3]` (MOS temperature); pause if $> 70^\circ\text{C}$.